Introduction
Part 1
Data Structure Algorithm

Ashim Khadka

Master Program

Outline

1. Algorithm
   - Correctness
2. Models of Computation
3. Analysis of Algorithms
   - Asymptotic Analysis
4. Brute Force Algorithm
5. Data Structure
   - Operation
   - Classification
   - Elementary Data Structure

Algorithm

- well-defined computational procedure that takes a set of values as input and produces a set of values as output
- step-by-step finite sequence of instructions, to solve a well-defined computational problem
  - procedure mapping each input to a single output (deterministic)
  -
- procedures used to efficiently store, retrieve, process, and manipulate the data organised in various data structures
- data structure (an array, linked list, or tree) provides a framework for storing and organizing data
  - algorithm defines how operations are performed on that data

Functions of Algorithms

1. Searching: Finding a specific item within a data structure (e.g., linear search, binary search)
2. Sorting: Arranging items in a specific order, such as numerical or alphabetical (e.g., bubble sort, merge sort, quick sort)
3. Insertion: Adding new data into a data structure
4. Updating: Modifying an existing item
5. Deletion: Removing an existing item from a data structure

Properties of a Good Algorithm

- Clear and Unambiguous: Each step must be precise, with only one interpretation possible
- Well-Defined Inputs: The algorithm should have zero or more clearly specified inputs
- Well-Defined Outputs: It must produce one or more outputs that match the desired result
- Finiteness: The algorithm must terminate after a finite number of steps and not run endlessly
- Feasibility: It should be achievable with the available resources (time and memory)
- Language Independent: The instructions are general and can be implemented in any programming language

Correctness

- Programs/algorithms have a **fixed size**, so how to prove correct?
- For small inputs, can use case analysis
- For arbitrarily large inputs, algorithm must be **recursive** or loop in some way
- Must use induction
  - Induction: domino effect of logic
  - An algorithm works for an input of size 1, then size 2, then size 3 (which would take forever)
  - Instead of prove two things:
    1. Base Case: The first domino falls (the algorithm works for the smallest possible input)
    2. Inductive Step: If any given domino falls, it must knock over the next one.
  - If you prove those two, you've effectively proven the algorithm works for every input size until the end of time

Recursive Algorithm

- technique used to solve computational problems by breaking them into smaller instances of the exact same problem
- two main components to prevent infinite loops and manage memory effectively
  - Base Case: handles the smallest possible input
    - serves as the stopping condition
    - Without a base case, the function would call itself indefinitely: runs out of memory
  - Recursive Step: reduces a large problem into a smaller version of itself
    - Each recursive call must modify the input in such a way that it moves closer to the base case

Example: Factorial $n! = n \times (n-1)!$

- Base Case: Stop condition $0! = 1$
- Recursive Step: The part where the function calls itself, but with a smaller number $(n-1)$
  - ensures we are always moving toward the base case

Calculating $3!$

Phase 1: The "Unwinding" (Expansion)

- computer can't solve 3! immediately, so it breaks it down:
- Call 3!: "calculate $3 \times 2!$"
  - puts $3\times?$ on hold and opens a new task for 2!
- Call 2!: "calculate $2 \times 1!$"
  - puts $2\times?$ on hold and opens a new task for 1!
- Call 1!: "calculate $1 \times 0!$"
  - puts $1\times?$ on hold and opens a new task for 0!
- Call 0!: Base Case Reached! The computer returns 1.

Phase 2: The "Winding" (Substitution)

- Back to $1!, 2!, 3!$
- Result: 6

Recursion manages this by using a **Call Stack**

Example: Fibonacci Sequence

- each number is the sum of the two preceding ones
- $0, 1, 1, 2, 3, 5, 8, \cdots$

Example: Fibonacci Sequence

- each number is the sum of the two preceding ones
- $0, 1, 1, 2, 3, 5, 8, \cdots$
- Base Case 1: $F(0) = 0$
- Base Case 2: $F(1) = 1$
- Recursive Step: $F(n) = F(n-1) + F(n-2)$

Example: Fibonacci Sequence

- each number is the sum of the two preceding ones
- $0, 1, 1, 2, 3, 5, 8, \cdots$
- Base Case 1: $F(0) = 0$
- Base Case 2: $F(1) = 1$
- Recursive Step: $F(n) = F(n-1) + F(n-2)$

The "Summing Up" Phase:

- $F(2) = F(1) + F(0) \rightarrow 1 + 0 = \mathbf{1}$
- $F(3) = F(2) + F(1) \rightarrow 1 + 1 = \mathbf{2}$
- $F(4) = F(3) + F(2) \rightarrow 2 + 1 = \mathbf{3}$

Models of Computation

- Machine-independent algorithm design depends on a hypothetical computer model: Random Access Machine (RAM)
- All memory is equally expensive to access
  - does not account for the memory hierarchy
- instructions are executed sequentially with no concurrent operations
- each simple operation takes exactly one time step
- primarily utilizes integer and floating-point data types

Analysis of Algorithms

How much stuff (time and memory) do I need as my input gets massive?

- Analyzing an algorithm isn't just about speed
- about finding the balance between
  - how easy it is to write (simplicity)
  - how sure we are that it works (correctness)
  - how much of computer's resources it consumes (time and space complexity)

Types of Analysis

1. **Worst case**
2. **Average case**
3. **Best case**

Worst Case

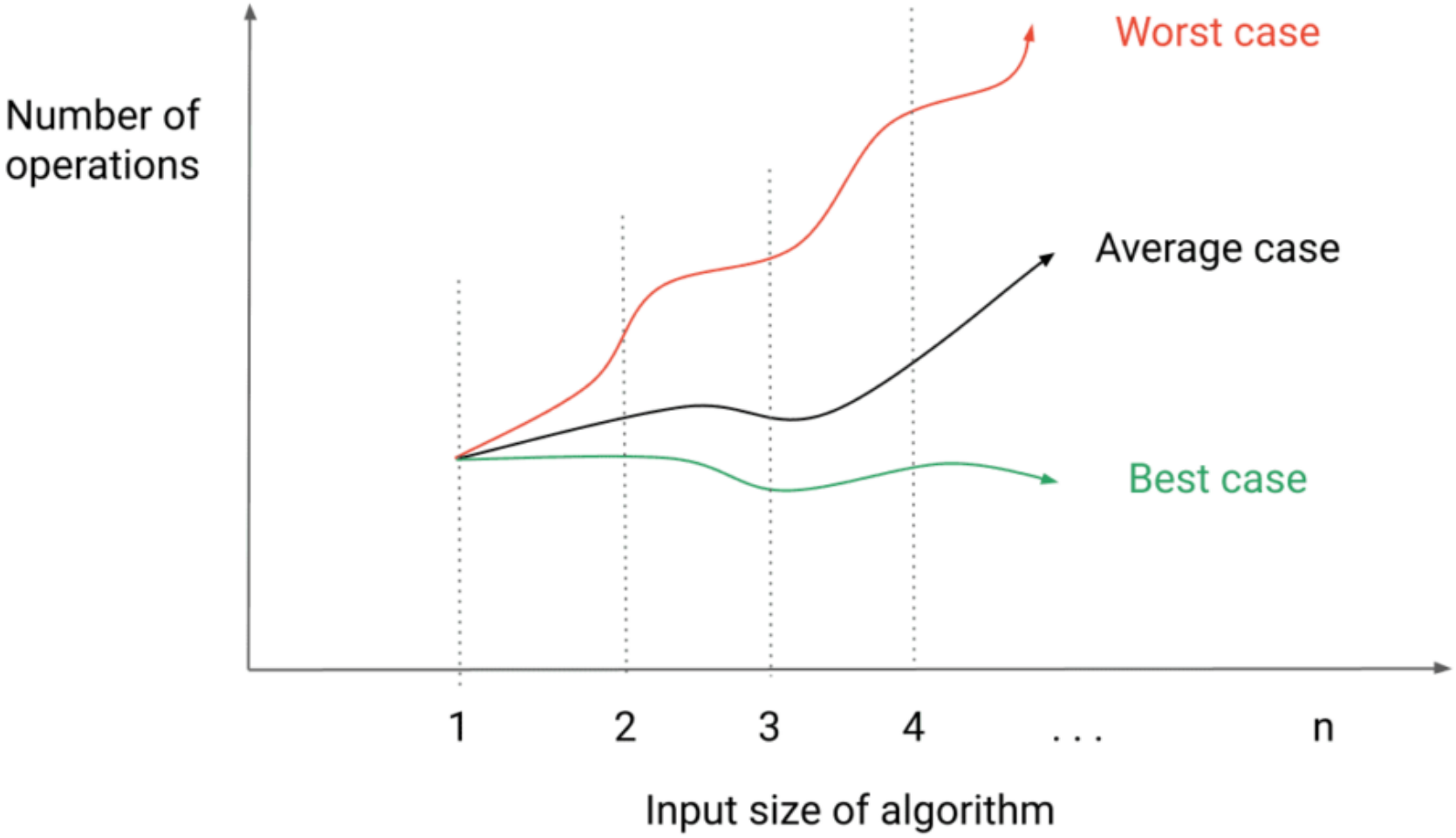
- absolute maximum steps: algorithm will take for an input of size n
- provides a "guaranteed" upper bound
- guarantee that the item does not occur in data
- Big-O notation (O)
- **Example:** Insertion Sort
  - list is provided in reverse order ([1, 2, 3]) but the input is [3, 2, 1]

Best Case

- minimum steps: rarely happens in the real-world environment
- occurs when the input instance is perfectly for the specific steps of the algorithm
  - Big-Omega (Ω) notation to describe lower bound
- In Sorting: For algorithms like Insertion Sort, the best case is an array that is already sorted.

Average Case

- expected steps for random data
- measure of how an algorithm will perform in the real world


Number of operations
Worst case
Average case
Best case
1
2
3
4
. . .
n
Input size of algorithm

Complexity: Time vs. Space

- Algorithm: light-fast but require more RAM than you have, or it can be tiny but take years to finish

1. Space complexity
2. Time complexity

Space Complexity

- amount of memory it needs to run to completion
- memory is a bounded resource that must be used wisely
  - especially when dealing with large datasets that may not fit in a computer's main memory
- total space required by a program is divided into the following components:
  1. Instruction Space: fixed amount of memory required to store the executable version of the program
  2. Data Space: stores constants and variable values
     - Fixed Space: Memory used for constants and simple variables
     - Structural Space: Memory for fixed-sized structural variables like arrays and structures
     - Dynamically Allocated Space: Space typically varies during execution
  3. Environment Stack Space:
     - space is used to store information necessary to resume suspended or partially completed functions

Time Complexity

- measures how the running time of an algorithm grows with input size n
- actual execution time depends on several factors such as:
  - Implementation of the algorithm
  - Programming language used
  - Compiler optimization techniques
  - CPU speed
  - Hardware characteristics
- Analyzing an algorithm to obtain its exact execution time: machine-dependent
  - estimate the order of growth of an algorithm with respect to input size n
  - estimation: **asymptotic analysis**
- time complexity: depends on the amount of input data
  - input size increases, the time required by the algorithm also increases
  - goal is to study how fast the running time grows as a function of input size, not the exact running time

Reasons for Studying Time Complexity

- helps to determine in advance whether an algorithm or program can provide a satisfactory real-time response
- For a given problem, multiple algorithms with different time requirements
  - time complexity helps in choosing the most efficient one
- enables fair comparison of algorithms independent of hardware and programming language
- helps in predicting the scalability of an algorithm for large input sizes

Asymptotic Analysis

- study of an algorithm's performance as the input size n becomes very large
- how fast an algorithm grows with input size
- provides a machine-independent way to compare algorithms

Goals of Asymptotic Analysis

- To estimate the performance of algorithms for large input sizes
- To compare different algorithms solving the same problem
- To predict scalability
- To ignore constant factors and lower-order terms

Rules of Asymptotic Analysis

1. Ignore constants:

$$O(5n) = O(n)$$

2. Ignore lower-order terms:

$$O(n^2 + n) = O(n^2)$$

3. Consider dominant term only

Types of Asymptotic Analysis

- Big-O notation (O): Worst case
  - Maximum time taken for any input
  - $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

  - Algorithm will not grow faster than $g(n)$
- Big-Omega (Ω): Best case
  - Minimum time taken for a given input
  - $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq c \cdot g(n) \leq f(n) \quad \forall n \geq n_0$$

  - Algorithm will take at least this much time
- Big-Theta (Θ): Average case
  - Expected time over all possible inputs
  - $f(n) = \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall n \geq n_0$$

  - Algorithm grows exactly like $g(n)$

Brute Force Algorithm

- all possible solutions and pick the correct one
- selecting the one that satisfies the problem condition

Traveling Salesman Problem (TSP)

- Given n cities and the distance between every pair of cities, find the shortest possible route that:
  - Visits each city exactly once
  - Returns to the starting city
  - Choose the minimum cost tour

Algorithm (Steps)

- Fix a starting city (say city 1)
- Generate all permutations of remaining cities
- For each permutation:
  - Calculate total tour cost
  - Keep track of minimum cost
- Output the minimum cost tour

Example: TSP

Table: Placeholder Caption

	A	B	C
A	0	10	15
B	10	0	20
C	15	20	0

- Minimum cost $= 45$
- Time complexity $= n!$

Data Structure

- primary purpose of most computer programs: storage and retrieval of information
- Data structure: specific way of organizing and storing data in a computer
  - can be accessed and modified efficiently
- **Think library:**
  - books: just thrown in a pile, finding *Introduction to Algorithms* $\rightarrow$ take forever
  - organized by genre and author: the structure, find it in seconds
- Data structure: ways to arrange data so that data items can be used efficiently in the main memory
  - serves as a mathematical or logical model that helps programmers understand the relationships between different data elements
- commonly used: strings, lists, trees, and graphs

Linear Data Structure

- structures form a sequence
- Traversal: can visit every element in a single pass
- Contiguous Memory: Arrays store elements in consecutive memory locations
- Non-Contiguous Memory: Linked Lists are linear in logic but not in memory; nodes are scattered, connected only by pointers

Non-Linear Data Structure

- used when data is too complex for a straight line
- Hierarchical (Trees): Tree structure has a parent-child relationship
  - An element can have multiple successors (children) but usually only one predecessor (parent)
- Network (Graphs): A Graph is the most flexible
  - any element can have multiple predecessors and multiple successors, forming a web or network

Data Structure Operation

- data appearing in data structure: processed by means of certain operations
- choice of a data structure depends on which operations you need most often
  - don't use every tool for every job, but knowing how each one works is what makes you a master craftsman (a great programmer)
- play a major role in processing of data:

1. **Creating**
   - initial operation required to declare and initialize a data structure
   - involves declaring the variable and reserving space in the computer's memory
   - Example: You buy a new phone and the OS initializes an empty database or array to hold your contacts
     - int contactList[100]; (This reserves space for 100 integers)
2. **Inserting**
   - adding new records or data items to the existing structure
     - In a stack, use the push operation to add a new item to the top of the stack
     - In a queue, the enqueue operation is used to add an element to the back of the line
   - Example: meet a new friend and hit "Save" on their name and number

3. **Deleting**
   - removes a record from the structure, often involving the reorganization of remaining elements or pointers
     - Contiguous structures, this often requires "shifting" other data to fill the hole
   - delete number: The list doesn't just leave a blank space; it shifts the names below them "up" to keep the list continuous
4. **Updating**
   - process of changing the data values of an element already residing within the data structure
     - without changing location of Data
   - Example: Your friend gets a new phone number
     - You don't delete the whole contact; you just edit the "Phone Number" field
5. **Traversing**
   - accessing each record exactly once so that the information can be processed or displayed
   - frequently referred to as "visiting" the records
   - Example: You scroll through your entire contact list from A to Z just to see how many people you know

6. **Searching**
   - involves finding the location of a specific record based on a given key or identifying all records that meet certain criteria
   - Example: You type "Mom" into the search bar
     - algorithm looks through the structure to find the memory address where "Mom" is stored
7. **Sorting**
   - process of arranging data elements into a specific logical order: Alpha, Numeric, Chronological in ascending or descending
   - Example: Your phone automatically sorts your contacts alphabetically by last name so they are easier to find later
8. **Merging**
   - act of combining elements from two different sorted sets into a single, larger sorted set
   - Example: You sync your Google Contacts with your Outlook Contacts
     - phone merges both lists into one master alphabetical list

9. Destroying

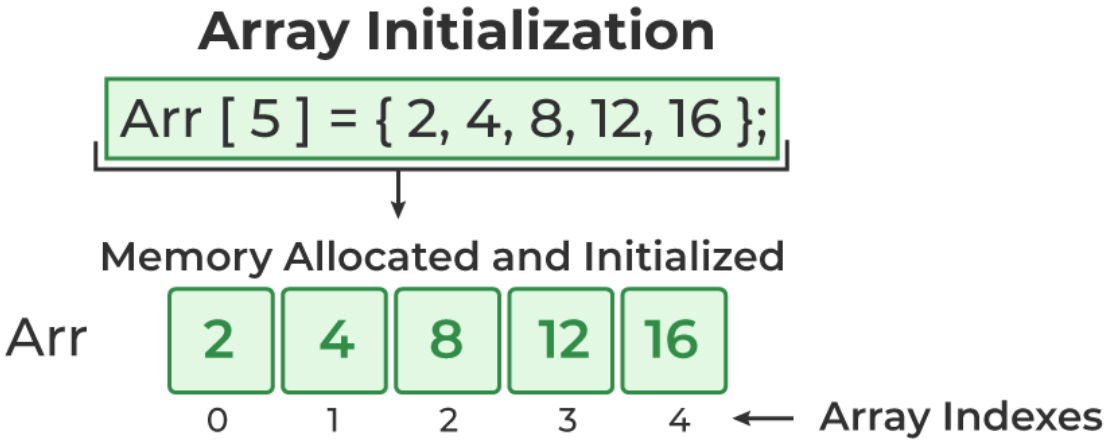
- cleanup phase: applied to a data structure when it is no longer needed
- ensure that the memory it occupies is released back to the system
- Example: "Factory Reset" on your phone
  - memory locations that held your contacts are wiped and marked as "available"

Classification of Data Structure

1. Contiguous
   - consists of single slabs of memory: based on arrays
   - array, matrices, heaps, and hash tables
2. Linked
   - made of multiple distinct chunks of memory joined by pointers
   - lists, trees, and graph

Array

- fixed-size structures where elements share the same data type and can be randomly accessed instantly via an index



- if the first element starts at memory address 1000 and each element takes up 4 bytes, the second element is guaranteed to be at 1004, the third at 1008, and so on
- computer doesn't have to "search" for an element; it uses a simple mathematical formula to jump straight to it:

$$Address[i] = BaseAddress + (i \times SizeOf\ \ Element)$$

- Searching is faster as elements are in continuous memory locations
- Memory management is taken care of by the compiler itself

Limitations

- Number of elements must be known in advance
- Inserting and Deletions are costlier as it involves shifting the rest of the elements

Stack and Queue

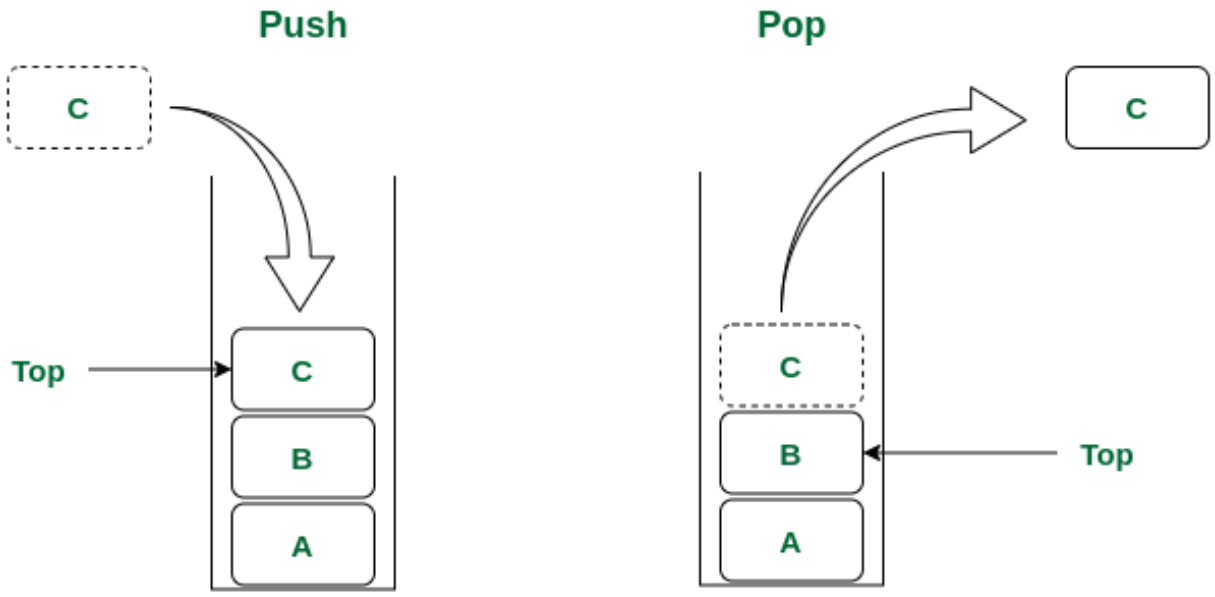
- traffic rules that govern how data enters and leaves
- both linear data structures
- structures serve as appropriate containers when the order of arrival matters

Stack

- linear data structure that follows the Last-In, First-Out (LIFO) principle

Operations:

- Push: Inserts an item at the top of the stack
- Pop: Removes and returns the top item
- Peek (or Top): Returns the top element without deleting it
- isEmpty: Checks if the stack contains no elements
- isFull: Checks if the stack has reached its allocated capacity



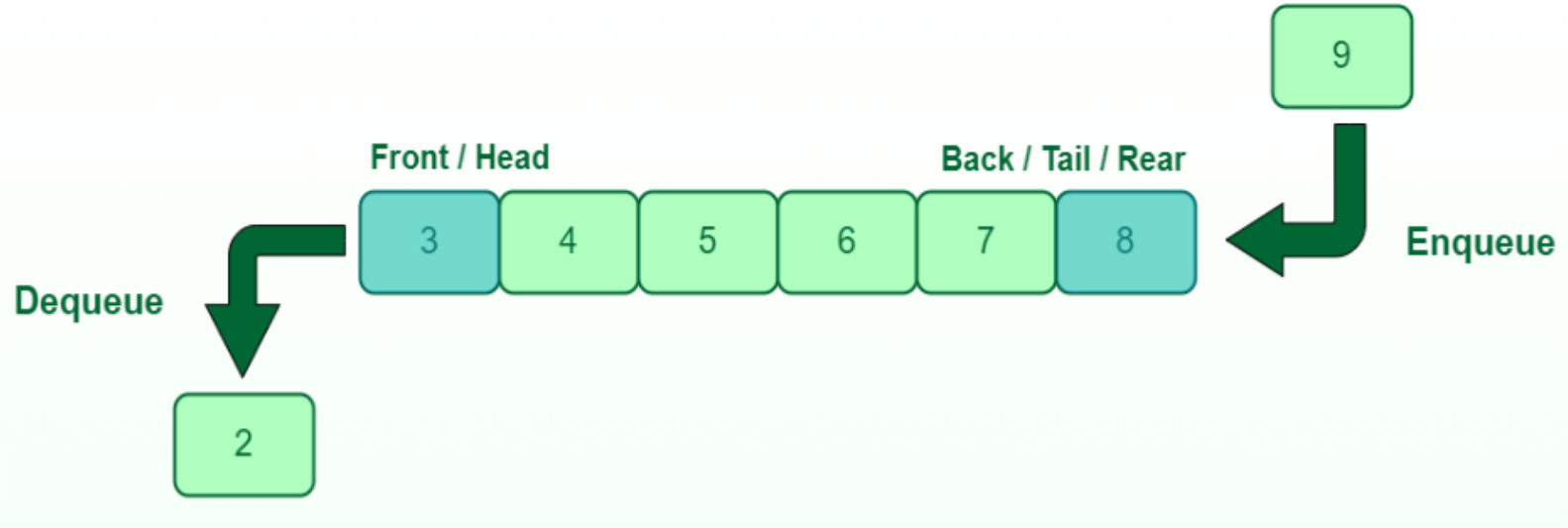
Stack Data Structure

Queue

- linear data structure that operates on a First-In, First-Out (FIFO) principle

Operations:

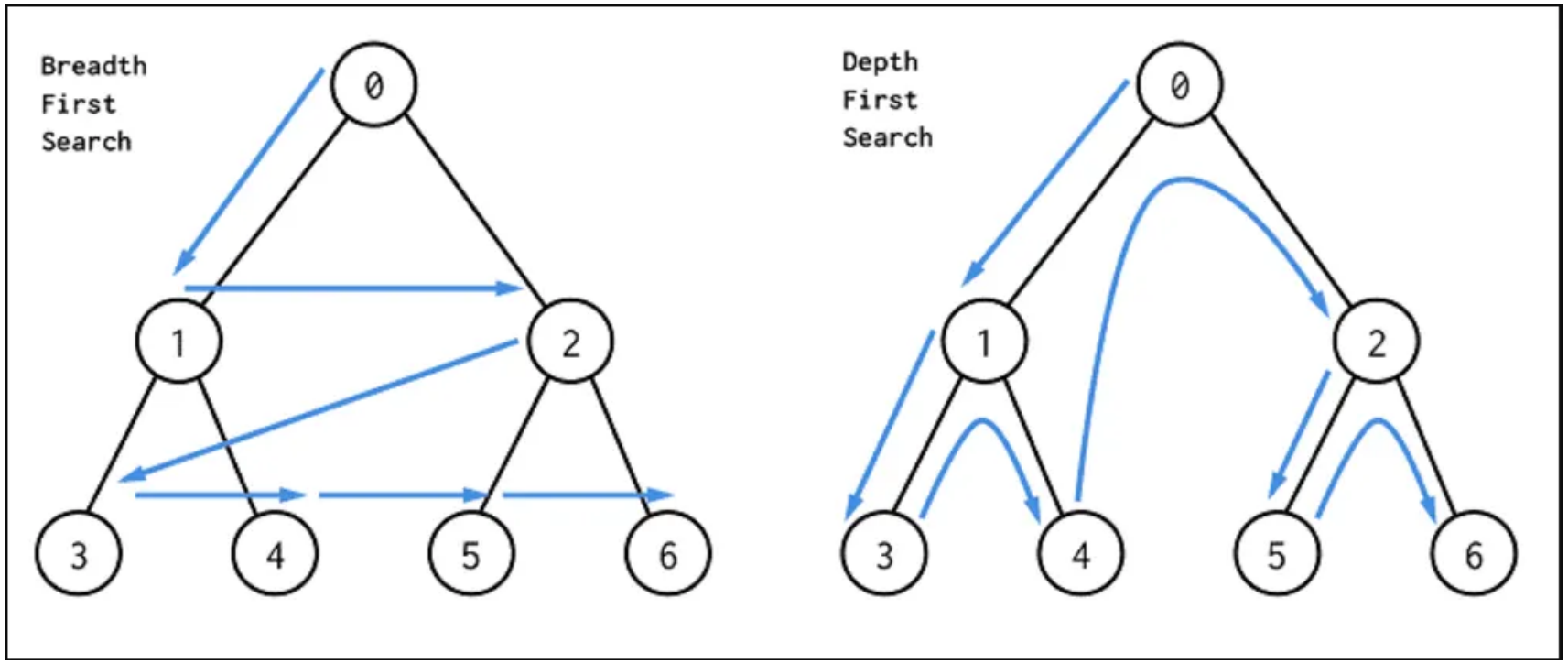
- Enqueue: Adds (or stores) an element to the end of the queue
- Dequeue: Removal of elements from the queue
- Peek (or front): Acquires the data element available at the front node of the queue without deleting it
- rear: operation returns the element at the rear end without removing it
- isFull: Validates if the queue is full
- isNull: Checks if the queue is empty



Queue Data Structure

Example

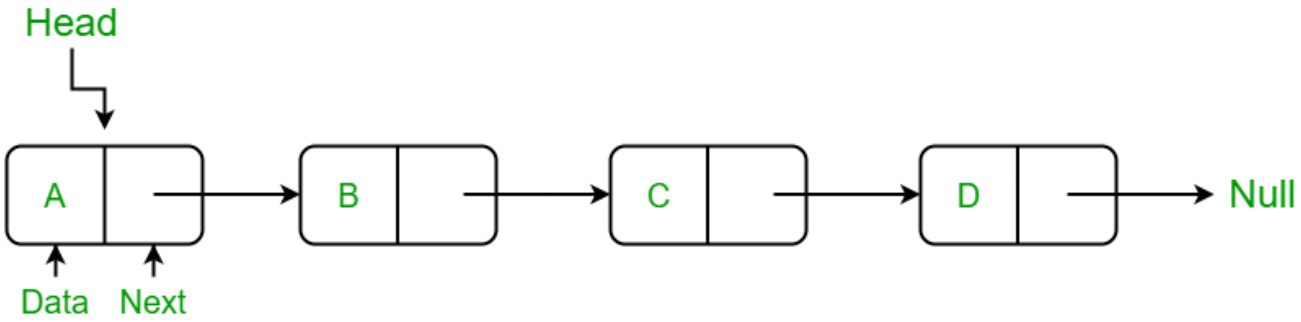
- printer spooling, Breadth-First Search (BFS)



Depth-First Search (DFS) = ?

Linked Lists

- allows to add elements as needed and delete elements without having to reshuffle the entire data structure
  - non-contiguous
- elements are arranged in an order determined by pointers
  - each element "points" to the next one
- composed of distinct chunks of memory called nodes
- Each node typically consists of a key (the data being stored) and a pointer to the next node in the sequence



Tree

- non-linear data structure
- mainly used to represent data containing a hierarchical relationship between elements like family tree, organisation chart
  - unique predecessor (parent) and potentially multiple successors (children)
- set of nodes and the connections between them, known as edges
  - Edges: 1-to-1 relationship between nodes and the edges

Basic Terminology

- Root: unique top-level node that has no parent
- Leaf: terminal node with a degree of zero (no children)
- Height: maximum level of any node in the tree
- Depth: length of the path from the root to that node
  - Each edge adds 1 unit of length to the path
  - Depth vs. Height: Root has a Depth of 0 but the maximum Height
    - Leaf has a Height of 0 but a high Depth
- Degree: total count of subtrees attached to that node

Types of Tree

- **Binary tree:** Every node has a maximum of two children, usually called Left and Right
  - Decision Tree: "Is it raining?" (Yes -> Take Umbrella / No -> Wear Sunglasses)
- **Ternary Tree:** node can have a Left, Mid, and Right child
- **N-ary Tree / Generic Tree:** no limit to how many children a node can have
  - Instead of "Left" and "Right" pointers, each node usually contains a List or Array of pointers to its children